

TP2 — Git en distant

Collaborer sur GitHub : branches, travail en équipe et Pull Requests

Objectifs de ce TP

Vous avez appris les commandes de base de Git en local. Pour pouvoir collaborer avec d'autres développeurs, il faut maintenant stocker votre dépôt sur un serveur distant, apprendre à travailler avec des branches et suivre un flux de travail proche de celui utilisé en entreprise.

À la fin de ce TP, vous serez capable de :

- Créer et sécuriser un compte GitHub
- Connecter un dépôt local à un dépôt distant via SSH
- Ignorer certains fichiers avec .gitignore
- Créer, faire évoluer et fusionner des branches
- Collaborer à plusieurs sur un même projet
- Résoudre un conflit de fusion
- Proposer et fusionner une Pull Request sur GitHub

Différents hôtes existent

GitHub n'est pas le seul hébergeur de dépôts Git : GitLab, Bitbucket ou Buddy proposent des services équivalents. Nous utiliserons **GitHub**, le plus répandu, mais les commandes Git restent identiques quel que soit l'hébergeur.

Exercice 1 — Créer un compte GitHub et le sécuriser

1. Si vous n'avez pas encore de compte, rendez-vous sur <https://github.com/> et créez un compte.

Un repository (dépôt) peut contenir votre code, mais aussi tout autre fichier que vous choisissez d'y stocker. Pour un dépôt public, restez prudent : tout ce qui y est stocké est accessible à tous, donc public.

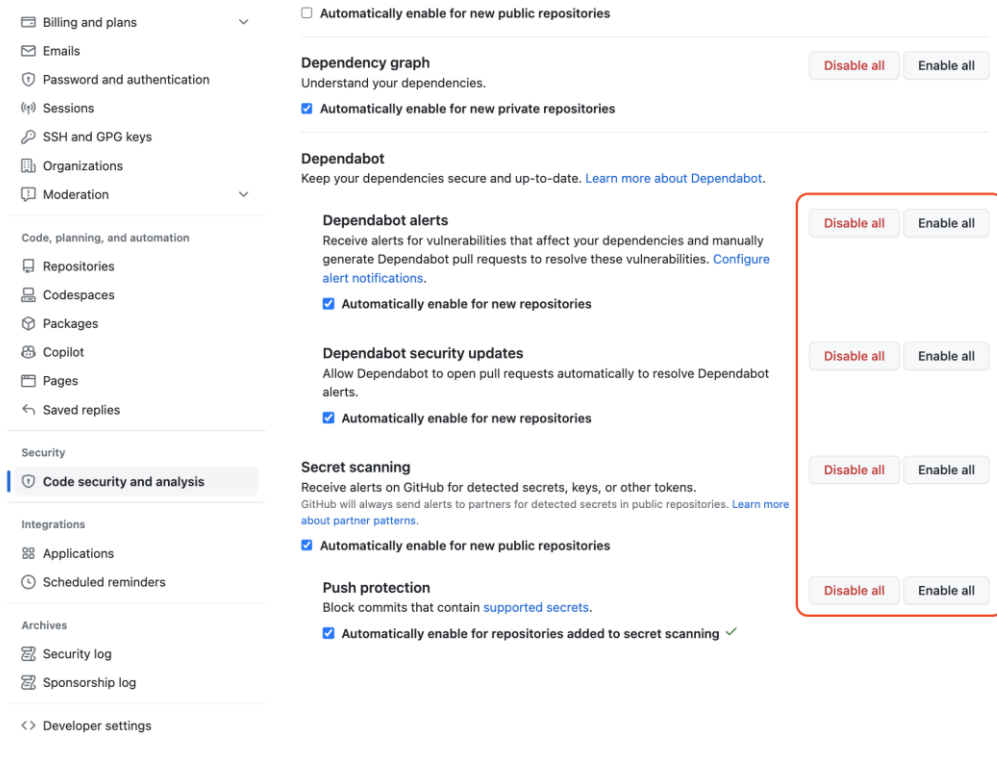
⚠ Attention aux informations sensibles

Il arrive fréquemment que des utilisateurs poussent par erreur des informations privées dans un dépôt : mots de passe, clés d'API, identifiants, numéros de carte bancaire... Une fois publiées, ces informations doivent être considérées comme compromises, même si le fichier est supprimé ensuite (l'historique Git les conserve).

Activer la Push Protection

GitHub propose une fonctionnalité appelée Push Protection : lorsque vous envoyez du code, GitHub le scanne pour vérifier qu'il ne contient aucune information sensible (clé, secret, token). Si un secret est détecté, l'envoi est bloqué avant même d'atteindre le serveur.

2. Cliquez sur votre photo de profil, puis **Settings** → **Code security and analysis**. Vous devriez obtenir la page suivante :



3. Cliquez sur Enable All, ou activez chaque option individuellement comme sur la capture ci-dessus. L'option sera automatiquement activée pour tout nouveau dépôt.

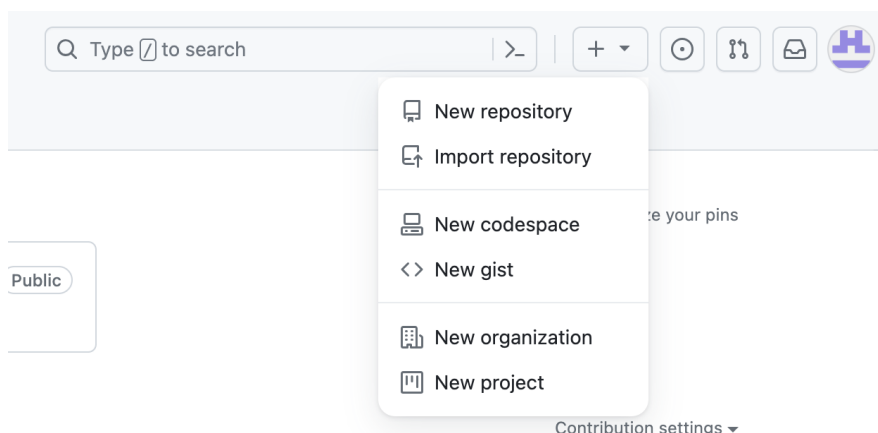
Activer l'authentification à deux facteurs (2FA)

Bonne pratique moderne

GitHub impose progressivement la 2FA à de nombreux comptes. Activez-la dès maintenant : **Settings** → **Password and authentication** → **Two-factor authentication**. Une application comme Google Authenticator ou 1Password suffit — évitez si possible la 2FA par SMS, moins sûre.

Exercice 2 — Créer un repository et un .gitignore

4. Cliquez sur le + en haut à droite de GitHub pour faire apparaître l'option *New repository*.



5. Nommez votre dépôt **GitExo1**, mettez-le en **public**, et **ne cochez pas** l'ajout d'un README.

Pourquoi et comment utiliser .gitignore

Certains fichiers ne doivent jamais être versionnés : dépendances installées, fichiers de configuration contenant des identifiants, fichiers temporaires générés par votre éditeur... Les envoyer sur GitHub alourdit inutilement le dépôt et peut exposer des informations sensibles.

La bonne pratique consiste à créer, avant le premier commit d'un projet, un fichier nommé exactement .gitignore à la racine du projet.

6. Si vous ne l'avez pas fait à la fin du TP1, créez un fichier .gitignore à la racine de GitExo1.
7. Listez-y les fichiers et dossiers à exclure du suivi de version, un motif par ligne. Par exemple, pour un projet web classique :

```
# Dépendances
node_modules/
vendor/

# Fichiers d'environnement / secrets
.env

# Fichiers générés par l'éditeur ou l'OS
.DS_Store
*.log
```

Astuce

GitHub propose des modèles de .gitignore prêts à l'emploi selon votre langage (Node, PHP, Java...) au moment de la création du dépôt, ou sur <https://github.com/github/gitignore>.

Un fichier déjà suivi par Git n'est pas automatiquement ignoré rétroactivement : .gitignore n'agit que sur les fichiers pas encore ajoutés à l'index. C'est pourquoi il est très important de le créer avant votre tout premier commit.

Exercice 3 — Connecter le dépôt local au dépôt distant

Pour envoyer votre dépôt local GitExo1 sur GitHub, il faut d'abord "relier" les deux dépôts, afin que Git sache où envoyer les modifications. Cette connexion s'authentifie par une clé SSH.

Remarque

Il n'est plus recommandé d'utiliser votre identifiant/mot de passe GitHub pour un clone/pull/push en ligne de commande : utilisez l'authentification par clé SSH autant que possible.

Générer une paire de clés SSH

8. Générez une clé SSH avec l'algorithme **ed25519** (remplacez l'adresse mail par celle de votre compte GitHub) :

```
ssh-keygen -t ed25519 -C "nom.prenom@lmdsio.fr"
```

9. Validez l'emplacement de sauvegarde par défaut (Entrée), puis choisissez une phrase de passe (passphrase).

```
PremierProjet git:(main)
→ ssh-keygen -t ed25519 -C "johndoe@example.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/Users/kassandre/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /Users/kassandre/.ssh/id_ed25519
Your public key has been saved in /Users/kassandre/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:1kDE2KZ/9P9AXwWNNxn5zt2yh8MN5esB71jtDkG+960 johndoe@example.com
The key's randomart image is:
+--[ED25519 256]--+
|      =o      .o.|
|      ..+     .+.|
|      o.      ..*|
|      . o.   o o=|
|      .S...  .+=+|
|      .. .  ..+=+|
|      .      o+X=|
|      .      X**|
|      .      .E0*|
+-----[SHA256]-----+
```

10. Copiez la clé publique dans le presse-papier :

```
# Windows (Git Bash)
clip < ~/.ssh/id_ed25519.pub

# macOS
pbcopy < ~/.ssh/id_ed25519.pub
```

11. Sur GitHub : **Settings** → **SSH and GPG keys** → **New SSH key**, donnez un titre, collez la clé copiée, puis **Add SSH key**.

Public profile

Account

Appearance

Accessibility

Notifications

Access

Billing and licensing

Emails

Password and authentication

Sessions

SSH and GPG keys

Organizations

Enterprises

Moderation

Add new SSH Key

Title

Key type

Authentication Key

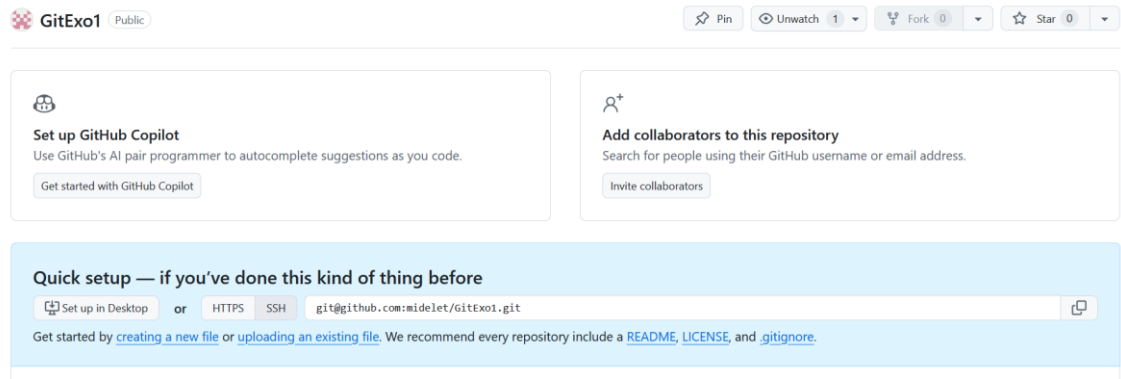
Key

Begins with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'

Add SSH key

Relier le dépôt local et pousser le premier commit

12. Sur la page de votre dépôt GitHub, dans « Quick setup », sélectionnez l'onglet SSH et copiez l'URL `git@github....`:



13. Dans Git Bash, placez-vous dans votre dépôt local GitExo1 et exécutez (en remplaçant le lien par le vôtre) :

```
git remote add origin git@github.com:votre-compte/GitExo1.git
git branch -M main
git push -u origin main
```

Actualisez (refresh F5) la page GitHub : votre version locale du projet est maintenant hébergée dans le cloud.

Pour synchroniser vos futures modifications, trois commandes suffiront : git add, git commit, git push.

Exercice 4 — Modifier le code et mettre à jour le dépôt distant

14. Dans `index.html`, ajoutez en bas de la page la phrase "Site créé le" suivie de la date du jour.
15. Indexez le fichier, validez la nouvelle version, puis envoyez-la sur le dépôt distant :

```
git add index.html
git commit -m "Ajout de la date de création"
git push origin main
```

Exercice 5 — Créer et faire vivre une branche

16. Listez les branches existantes avec `git branch`. Vous ne devriez avoir qu'une branche `main`.
17. Votre site sera un site marchand ; vous allez développer un panier. Créez une branche `panier` et basculez dessus :

```
git switch -c panier
```

git switch, la commande moderne

Depuis Git 2.23, `git switch` remplace avantageusement `git checkout` pour changer de branche : plus explicite et moins risqué. L'option `-c` crée la branche et bascule dessus en une seule commande (équivalent de `git branch panier` suivi de `git switch panier`).

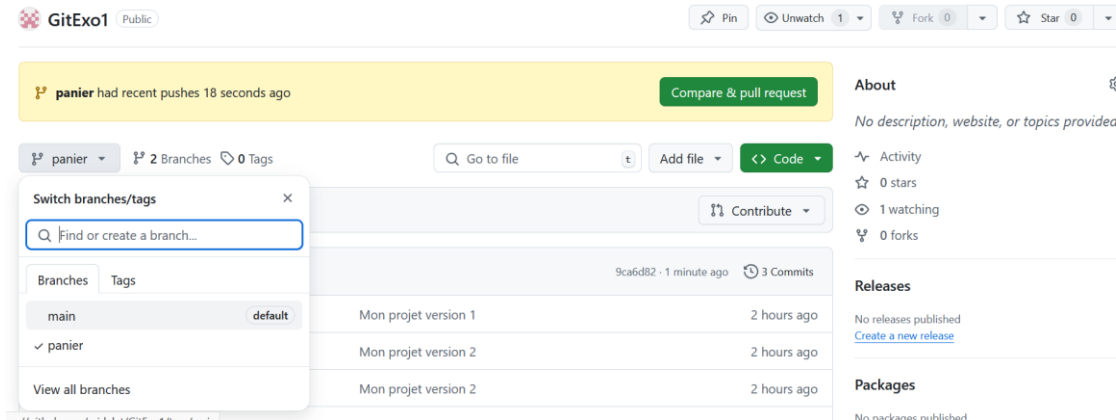
18. Vérifiez avec `git branch` que vous êtes bien sur `panier` (elle est repérable par une astérisque).
19. Créez une page `panier.html` affichant "Votre panier est vide". Dans `profil.html`, ajoutez un lien "Voir mon panier" vers cette page.
20. Indexez et validez ces modifications :

```
git add panier.html profil.html
git commit -m "Ajout du panier"
```

21. Envoyez la branche **panier** sur le dépôt distant :

```
git push origin panier
```

Sur GitHub, vous devriez voir apparaître la nouvelle branche, avec une proposition de créer une Pull Request :



Exercice 6 — Fusionner une branche dans main

Supposons que le développement de la fonctionnalité panier est terminé : il faut maintenant l'intégrer dans la branche principale. Revenez sur main puis fusionnez panier :

```
git switch main
git merge panier
```

Fusionner panier dans main ne fait pas disparaître la branche panier : elle continue d'exister et peut recevoir de nouveaux commits.

Il vaut mieux fusionner régulièrement plutôt qu'attendre : plus les branches divergent, plus les conflits deviennent nombreux et difficiles à résoudre.

Exercice 7 — Développer en équipe

1. Choisissez un binôme. Sur votre repo Github à vous, donnez accès en écriture à votre binôme : **Settings** → **Collaborators** → **Add people**, puis entrez son adresse mail. Il devra valider l'invitation reçue par mail.
2. Créez en local un dossier `GitExo1_nomBinome` (en remplaçant `nomBinome` par le nom de votre camarade).
3. Ouvrez le projet de votre binôme dans une nouvelle fenêtre VSCode, en maintenant votre projet personnel ouvert : **Fichiers** → **Nouvelle fenêtre** puis **Ouvrir un dossier** → **GitExo1_nomBinome**
4. Demandez le lien du repo Github GitExo1 de votre binôme. Depuis la page du dépôt GitExo de votre binôme, copiez le lien SSH puis dans un terminal sur VSCode, exécutez la commande suivante pour cloner le repo de votre binôme hébergé sur Github dans votre dossier `GitExo1_nomBinome` local (attention, remplacer le lien SSH ci-dessous et ne pas oublier le point à la fin) :

```
git clone git@github.com:nom-utilisateur-de-votre-binome/GitExo1.git .
```

Dans votre dépôt, branche main

5. Dans votre projet à vous, dans la branche main (vérifiez que vous y êtes avec la commande `git branch`, sinon changez avec `git switch main`), ajoutez une règle CSS pour un fond jaune, dans un fichier `style.css` que vous créez :

```
body {  
  background-color: yellow;  
}
```

6. Ajoutez la référence à ce fichier dans `index.html` avec la balise `<link>`, puis validez ces changements sur `main`.

Aide syntaxe HTML : Dans `index.html`, ajouter entre les balises `<head>` `</head>` la ligne suivante
`<link rel="stylesheet" href="style.css">`

Dans le dépôt de votre binôme, branche dev

7. Dans le projet de votre binôme, créez une branche `dev`, ajoutez une règle CSS pour un fond bleu dans `style.css`, référez le fichier dans `index.html`.
8. Testez, validez (add, commit), puis poussez la branche dev vers le dépôt distant :

```
git push origin dev
```

A ce stade, attendez que votre binôme ait terminé l'exercice 7 pour continuer.

Exercice 8 — Récupérer une branche distante et résoudre un conflit

9. Dans votre propre dépôt, listez les branches locales et distantes :

```
git branch -a
```

10. Récupérez la branche `dev` de votre binôme avec `git pull`, puis refaites `git branch -a` : la branche distante `origin/dev` doit apparaître.

```
git pull
```

11. Assurez vous d'être sur la branche main, puis fusionnez `origin/dev` dans `main` :

```
git merge origin/dev
```

Un conflit doit apparaître : vous avez mis un fond jaune, votre binôme un fond bleu sur la même ligne de `style.css`. Ouvrez le fichier avec votre éditeur :



Juste après `<<<<<< HEAD` se trouve la version de la branche pointée par HEAD (le dernier commit de main). La séquence `=====
>>>>>>` marque la fin de cette version et le début de celle d'`origin/dev`. La séquence `>>>>>>`

origin/dev marque la fin de la zone en conflit. En résumé, la version en haut correspond à votre travail, la version en bas correspond au travail de votre collègue.

12. Éditez le code à la main : conservez la couleur de votre choix et retirez toutes les marques de conflit. Par exemple :



13. Validez la fusion (un message de commit par défaut est proposé, vous pouvez le conserver) :

```
git add style.css
git commit
```

14. Poussez votre version fusionnée, puis vérifiez le résultat sur GitHub :

```
git push origin main
```

Exercice 9 — Proposer une Pull Request

Jusqu'ici, vous avez fusionné des branches directement en ligne de commande. En entreprise, on préfère généralement passer par une Pull Request (PR) : une proposition de fusion faite sur GitHub, qui permet à un collègue de relire le code avant de l'intégrer.

Pourquoi « Pull Request » ? En fait, vous demandez au propriétaire du dépôt de tirer / récupérer votre code et de l'intégrer dans son dépôt.

15. Dans le dépôt de votre binôme, créez une nouvelle branche pour une petite amélioration, par exemple **ajout-footer**, ajoutez un pied de page à **index.html**, commitez, puis poussez la branche.

Ajoutez le code dans index.html puis exécutez les commandes suivantes :

```
git switch -c ajout-footer
git add index.html
git commit -m "Ajout du pied de page"
git push origin ajout-footer
```

16. Sur GitHub, cliquez sur **Compare & pull request** (voir capture de l'exercice 5). Renseignez un titre et une description expliquant le changement.
17. Dans **Reviewers**, demandez la relecture de votre binôme.
18. Votre binôme relit le code depuis l'onglet *Files changed*, laisse d'éventuels commentaires, puis approuve la Pull Request.
19. Une fois approuvée, cliquez sur **Merge pull request** pour l'intégrer dans **main**. Vous pouvez ensuite supprimer la branche **ajout-footer** (bouton proposé automatiquement), elle n'est plus utile une fois fusionnée.
20. En local, récupérez le résultat et supprimez votre copie de la branche fusionnée :


```
git switch main
git pull
git branch -d ajout-footer
```

Pourquoi c'est important

La Pull Request laisse une trace écrite de la discussion autour d'un changement, permet une revue de code avant intégration, et déclenche souvent des vérifications automatiques (tests, linter, que nous étudierons plus tard). C'est le flux de travail standard sur GitHub en entreprise.

Méthode — Une session de travail typique

Quand vous commencez à travailler sur un dépôt local, la première chose à faire est de vérifier que le dépôt distant n'a pas avancé pendant votre absence :

```
$ git fetch
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
Depuis github.com:votre-compte/GitExo1
   2974c4b..b9c5516  main -> origin/main

$ git status
Sur la branche main
Votre branche est en retard sur 'origin/main' de 1 commit.
(utilisez "git pull" pour mettre à jour votre branche locale)
```

git fetch récupère l'historique du dépôt distant sans le fusionner. git status indique ensuite si votre branche locale est en retard. S'il n'y a pas de conflit prévisible, un simple git pull met à jour votre copie locale :

```
$ git pull
Mise à jour 2974c4b..b9c5516
Fast-forward
 style.css | 1 +
1 file changed, 1 insertion(+)
```

Pour travailler sur une autre branche que main, il faut :

- basculer sur cette branche (git switch) ;
- refaire un git fetch si c'est une branche distante ;
- éventuellement un git pull si la branche distante a avancé.

Une fois le travail terminé : indexez vos fichiers, committez, et poussez avec git push (git add, commit, push).

Exercice 10 — Appliquer cette méthode à votre projet

Reprenez les exercices précédents en les appliquant à votre projet React ou Laravel. Vous pouvez vous aider du mémo Git fourni dans les ressources.

N'oubliez pas le .gitignore adapté (allez voir sur <https://github.com/github/gitignore>)

Pour un projet React : excluez `node_modules/` et `build/`. Pour un projet Laravel : excluez `vendor/`, `.env` et `storage/*.key`.

Si vous voulez disposer facilement de plusieurs versions de votre code, associez chaque version à une branche, puis changez de branche au moment de commencer la version suivante. Par exemple, pour votre projet React, vous pouvez créer une branche par séance de TP.

Pour aller plus loin

- [L'histoire de GitHub](#)
- [Wikipédia : logiciel de gestion de versions](#)
- GitHub Desktop et l'intégration Git de VS Code : deux interfaces graphiques utiles pour visualiser les diffs et l'historique, en complément de la ligne de commande.
- Conventional Commits : une convention de rédaction des messages de commit (`feat: ...`, `fix: ...`) de plus en plus utilisée en entreprise.